

中国民航大学

《软件工程课程设计》

软件设计规格说明书（修订版）



基于智能 Agent 的民航旅客起飞前行程规划与 延误风险预警系统

组长学号姓名： 240340204 陈源齐

成员学号姓名： 240340154 陈志恒

240340164 曹力文

240340167 李昊洋

240340195 吴启训

二〇二六年九月

目录

第 1 章 引言	1
1.1 软件设计目标和原则	1
1.2 软件设计的约束和限制	2
第 2 章 软件体系结构设计	2
2.1 关键控制流程	4
2.2 数据与错误流	4
2.3 架构质量属性	4
第 3 章 用户界面设计	5
3.1 系统界面的外观设计及其类表示	5
3.1.1 行程创建功能与前端接入	5
3.1.2 当前行程仪表盘界面	6
3.2 系统界面流设计	7
3.3 界面可用性与异常设计	8
第 4 章 详细设计	9
4.1 用例设计	9
4.1.1 “创建行程”用例实现的设计方案	9
4.1.2 “更新位置或手动阶段”用例实现的设计方案	10
4.1.3 “查询状态与实时订阅”用例实现的设计方案	11
4.2 类设计	13
4.2.1 DataAgent 详细设计	15
4.2.2 AdviceAgent 详细设计	15
4.2.3 Runtime 与并发控制	16
4.3 数据模型设计	16
4.3.1 航旅智行系统数据设计类图	16
4.3.2 系统数据的操作设计	18
4.3.3 枚举、时间与空值设计	19
4.4 部署设计	19

4.4.1 部署安全与运维.....	20
4.5 接口与契约实现要求.....	21
4.6 测试与验收设计.....	21
4.7 设计决策与后续扩展.....	22
5 集成测试与验证.....	23
5.1 构建与自动化测试.....	23
5.2 EOOB 航班抓取验证.....	23
5.3 UI 联调过程与截图.....	24
5.4 测试结论与已知问题.....	26

图目录

图 2-1 系统体系结构逻辑视图（当前实现）	2
图 3-1 行程创建功能的当前实现边界	3
图 3-2 当前行程仪表盘界面组成	3
图 3-3 行程仪表盘前端类与模块表示	3
图 3-4 行程仪表盘加载与实时更新顺序图	4
图 4-1 创建行程与异步分析顺序图	5
图 4-2 更新位置或手动阶段顺序图	5
图 4-3 查询快照与 SSE 订阅顺序图	5
图 4-4 后端实现设计类图	6
图 4-5 领域与契约数据类图	6
图 4-6 系统部署图（当前可运行链路 with 外部边界）	7
图 5-1 创建行程界面	24
图 5-2 行程分析处理中界面	25
图 5-3 EOOB 数据与 SSE 就绪界面	25
图 5-4 手动阶段确认后的界面	26

表目录

本说明书中的表格随所属章节内联排布，未单独编号。

第 1 章 引言

1.1 软件设计目标和原则

本设计以课程汇报 PPT 所确定的“旅程伴随式 Agent”目标、`docs/api/openapi.yaml` 冻结契约和 `aviation-journey-agent` 当前代码为依据，给出可直接指导后续编码、联调和测试的软件实现说明。本文将 PPT 中的目标能力与截至 2026 年 9 月已经落地的能力分开表述，所有实现类名、属性和方法名均以代码为准。

当前版本已经形成完整的可运行闭环：Vue 创建表单提交行程，Gin REST 接口受理，DataAgent 通过 EOOB 航班检索、状态和机场信息技能补全世界状态，AdviceAgent 生成建议或安全降级，MemoryStore 保存快照，SSEHub 实时推送；前端同时提供定位/手动阶段确认以及断线轮询回退。嵌入式地图和实时道路导航 ETA 尚未实现，UML 对这些边界作明确标注。

设计过程遵循以下原则：

- 契约优先原则。`docs/api/openapi.yaml` 是接口路径、状态码、字段、枚举和空值语义的唯一真源；人读文档、前端生成类型和后端响应都必须与其一致。
- 关注点分离原则。DataAgent 只负责数据获取、标准化和质量判断；AdviceAgent 只消费 WorldState 并生成建议；前端只渲染，不进行风险、缓冲和阶段等业务计算。
- 高内聚、低耦合原则。API、Runtime、Agent、Skill、Store 和 SSE 各自承担单一职责，通过稳定接口协作，避免跨层直接访问内部状态。
- 安全降级原则。未知值用 `null` 表达，数组无内容时返回空数组；外部数据源失败不得拖垮整个分析，关键数据不足时风险必须为 `unknown` 或至少 `yellow`。
- 可解释原则。建议依据、数据来源、更新时间和降级原因通过 `advice.reasons`、`state.quality` 和 `state.updatedAt` 呈现，避免把辅助建议包装成确定承诺。
- 用户确认原则。导航、联系航司等动作由用户主动触发；系统不自动执行改签、支付、叫车等高影响操作。
- 迭代与可扩展原则。v1 先完成首个航段的可靠闭环，通过 Skill 接口扩展真实数据源，后续再扩展多航段、持久化和通知服务。

1.2 软件设计的约束和限制

类别	约束或限制	设计影响
部署形态	后端为单个 Go 二进制；前端为 Vue 3 单页应用	不引入微服务、消息队列或独立模型服务
技术栈	Go 1.22、Gin、Vue 3、TypeScript、Pinia、Axios、Vite	实现应复用现有依赖，避免无必要新增框架
接口契约	统一前缀 /api/v1；4 个 Journey 接口；POST 异步返回 202	路径、状态码和字段以 OpenAPI 为准；当前 Go 校验尚未覆盖全部格式约束
状态存储	课程阶段采用进程内 MemoryStore，包含 journeys、snapshots、progress 三个映射	服务重启后数据丢失，不支持多实例共享状态
实时通信	只使用 SSE，不使用 WebSocket	服务端推送完整快照，客户端整体替换本地状态
数据来源	当前链路已接入 EOOB 航班检索、状态和机场页；路线 ETA 为本地低置信度估算	外部源不可用时返回 unknown/null 并解释原因
地图能力	不嵌入地图 SDK，不采集导航轨迹	城市导航使用高德 URI；机场内部使用文字指引和时间轴
业务边界	不自动改签、支付、购票或叫车	Action 是展示型建议，不是服务端命令
版本范围	flights[] 可包含多个航段；当前数据分析与界面重点使用首个航段	多航段衔接属于后续版本
隐私	定位仅在用户授权后上报，服务端使用自己的时钟	客户端不得上传当前时间，定位采集遵循最小化原则

系统受 EOOB 可访问性、模型调用、网络质量和定位精度限制。当前 FlightIdentitySkill 负责按航班号与日期解析完整航段，FlightStatusSkill 调用 EOOB 结构化状态接口，AirportStatusSkill 解析 EOOB 机场页，RouteETASkill 依据机场坐标和旅客位置作直线距离 $\times 1.3$ 、30 km/h 的低置信度估算。缺少可靠观测时字段保持 null 或 unknown。

第 2 章 软件体系结构设计

系统采用客户端/服务器结构。客户端是 Vue 3 + TypeScript SPA，由 Dashboard.vue、Pinia 状态模块、行程创建表单和八个展示/交互组件构成；服务端是

单个 Go 进程，集成 Gin API、runtime.Runtime、data.Agent、advice.Agent、skill.Registry、store.MemoryStore 和 runtime.SSEHub。数据技能使用带自洽浏览器提示的标准 HTTP 客户端访问 EOOB，模型服务只承担建议生成；未配置模型时系统按规则安全降级。



图 2-1 系统体系结构逻辑视图（当前实现）

逻辑结构按界面与接口、运行时编排、Agent 与技能、存储与推送、外部能力划分。图中只列当前代码存在的包、类型和公开入口，避免把 PPT 目标或尚未实现的类误写成现有实现。

层次/模块	当前主要职责	边界或尚未实现
Dashboard.vue	创建行程，组合状态条、新鲜度、指标、时间轴、行动卡片和阶段确认	尚未实现嵌入式地图；枚举中文化仍需完善
useJourneyStore	维护 snapshot/lastReady，建立 EventSource，20 秒后启用 15 秒轮询回退	内存态刷新页面后不保留当前 journeyId
api/journey.ts	createJourney、getState、updateLocation、createStream	不调用外部航班或地图数据接口
api.JourneyHandler	JSON 绑定、请求校验、状态码和 SSE 协议适配	不包含 Agent 规则
runtime.Runtime	保存 processing、异步分析、busy 去重、保存并发布完整快照	并发重算直接返回当前快照，不排队
data.Agent	航班身份补全、MissingFields、Registry.Query、Observation 校验、质量评估	不生成最终建议；单项外部失败记录 issue
advice.Agent	模型建议、结构清洗、手动阶段覆盖、风险下限	不访问外部航班/机场数据，不执行高影响操作
skill.Registry / Skill	调用 EOOB 航班检索、状态、机场技能及本地 RouteETA，并汇总 Observation/issue	EOOB 可用性影响新鲜度；RouteETA 非实时路况
MemoryStore / SSEHub	线程安全保存三类状态；按 journeyId 广播快照	无持久化、无跨实例共享

2.1 关键控制流程

创建行程时，`JourneyHandler.Create` 依次执行 `ShouldBindJSON` 和 `CreateJourneyRequest.Validate`，构造 `Journey` 后调用 `Runtime.CreateJourney`。`Runtime` 保存 `Journey` 与空 `JourneyProgress`，立即通过 `publishProcessing` 保存并广播 `processing` 快照，再由 `analyzeAsync` 启动最长 90 秒的后台分析；HTTP 层返回 202 `AnalysisAccepted`。

定位或手动阶段更新由 `JourneyHandler.UpdateLocation` 调用 `Runtime.ApplyLocation`。`lat` 与 `lng` 必须成对出现；坐标上报会清除旧 `ManualStage`，`stage` 上报会写入手动阶段。`Runtime` 同样先发布 `processing`，再异步执行 `Recalculate`。

`Runtime` 使用 `sync.Map` 字段 `busy` 对同一 `journeyId` 的并发分析去重。`Recalculate` 发现该行程正在分析时直接返回 `Store` 中的当前快照，不排队等待；不同 `journeyId` 仍可并行。该实现与“每行程互斥锁串行排队”不同，UML 按实际 `busy.LoadOrStore` 行为表示。

2.2 数据与错误流

`DataAgent` 从 `NewState` 开始，通过 `MissingFields` 计算待查询字段，调用 `Registry.Query` 汇总各 `Skill` 的 `Observation` 和 `issue`，再由 `applyObservations` 做类型、枚举和 RFC3339 校验。单个 `Skill` 失败只追加 `issue`；只有 `BuildState` 返回错误时 `Runtime` 才生成 `failed` 快照。

HTTP 层参数错误返回 400，行程不存在返回 404，未预期服务端错误返回 500。分析过程错误不通过已结束的 POST 请求返回，而写入 `status=failed` 与 `error`。前端保留已有内容，显示错误横幅和重新分析入口。

2.3 架构质量属性

- 可靠性：外部调用设置超时并支持部分成功；快照始终具有完整顶层结构。
- 实时性：位置变化完成重算后立即推送；无变化时每 15 秒发送 `heartbeat`。
- 一致性：`/state` 与 SSE `snapshot` 使用同一个 `JourneySnapshot` 类型，客户

端不做增量合并。

- 可维护性：前端类型由 OpenAPI 自动生成；新增数据源通过实现 Skill 接口接入。
- 可测试性：规则计算保持纯函数；mockserver 提供正常、延误和失败场景。
- 安全性：只采集必要定位，不记录导航轨迹，不自动执行外部业务操作。

第 3 章 用户界面设计

3.1 系统界面的外观设计及其类表示

课程 PPT 将客户端目标概括为状态、时间轴、地图、倒计时和行动卡片。当前前端已实现行程创建、状态与数据新鲜度、指标卡、时间轴、文字指引、行动卡片、外部导航按钮、定位及六类手动阶段确认；嵌入式地图仍未实现。界面类图按实际挂载关系给出组件 props、局部方法和状态依赖。

3.1.1 行程创建功能与前端接入

Dashboard.vue 已挂载行程创建表单，收集航班号、日期、出发/到达机场三字码和托运行李信息，并调用 api/journey.ts 的 createJourney(payload)。后端 JourneyHandler.Create 经 CreateJourneyRequest.Validate 后调用 Runtime.CreateJourney，返回 journeyId 和 processing 快照；前端随后调用 useJourneyStore.load(id) 进入分析与订阅流程。

来源	界面或模块	当前实现状态
PPT 目标	行程创建表单	已挂载；字段为航班、日期、机场和托运行李
已实现模块	api/journey.ts	createJourney 、 getState 、 updateLocation 、 createStream 均已接入
已实现视图	Dashboard.vue	提交后取得 journeyId，展示 processing/ready/failed 并支持重试
已实现模块	useLocation / StageConfirm	浏览器定位与六类手动阶段已接入视图
已实现后端	JourneyHandler.Create	校验非空字段，调用 Runtime，返回 202



图 3-1 行程创建功能的当前实现边界

3.1.2 当前行程仪表盘界面

Dashboard.vue 读取 useJourneyStore 的 snapshot，组合 StatusBar、FreshnessBar、MetricsRow、Timeline 和 ActionCardList。ActionCardList 在 action.nav 存在时渲染 NavigateButton；NavigateButton.open 先打开 nav.app，1.2 秒后回退 nav.web，并在页面进入后台时取消回退定时器。

Dashboard.vue 对 idle、processing、ready 和 failed 分别呈现创建表单、骨架屏、完整仪表盘和失败重试；处理期间保留 lastReady 以避免已有结果闪烁。risk=unknown 作为明确枚举传入 StatusBar，前端不得把它解释为 green。

Dashboard.vue 已组合 StageConfirm，并调用 useLocation 的 locate/confirmStage：浏览器定位成功时上报 lat/lng，定位失败时保留错误信息；旅客也可直接确认六种 ManualStage。每次上报返回 202 processing，后端重算完成后通过 SSE 更新页面。

图 3-3 分为两个子图：子图（a）列出 Dashboard 已挂载组件及其 props/局部方法；子图（b）列出 Pinia 状态、定位组合式函数、API 模块和契约类型之间的依赖。Vue props 和导出函数用“+”表示对外可见，组件内部 computed 或普通函数用“-”表示局部实现。

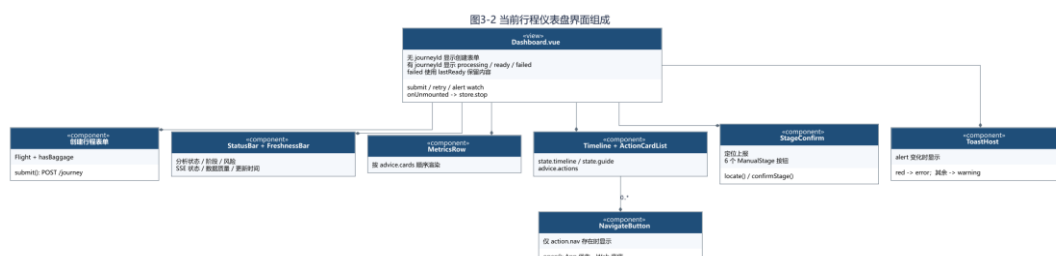


图 3-2 当前行程仪表盘界面组成

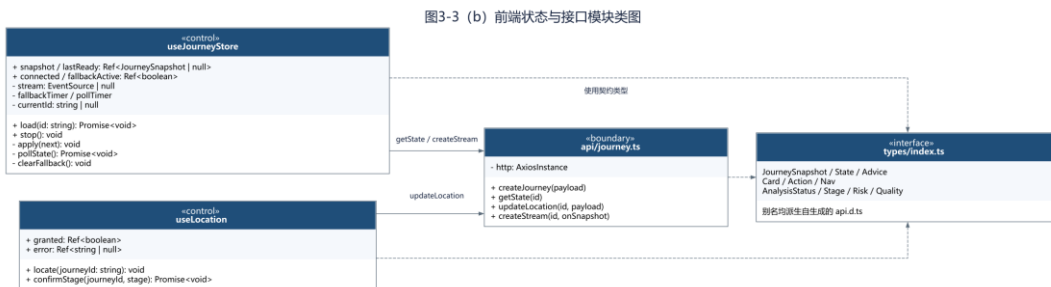
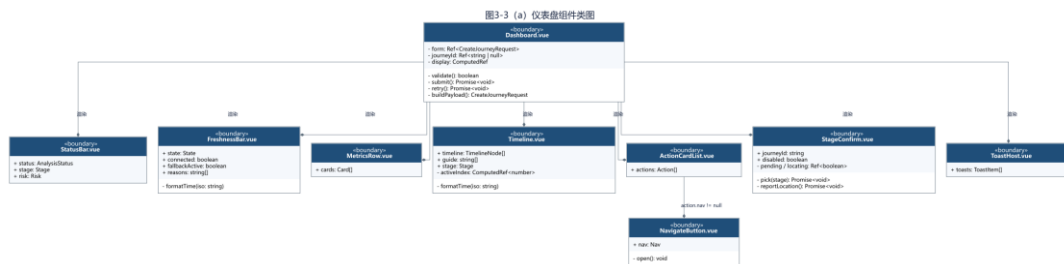


图 3-3 行程仪表盘前端类与模块表示

3.2 系统界面流设计

当前前端以 Dashboard.vue 作为单页入口。创建成功后组件直接取得 journeyId，调用 useJourneyStore.load(id)：先读取 /state，再建立 EventSource 订阅；提交失败可以保留输入并重试，页面卸载时调用 stop 清理连接和回退定时器。

SSE snapshot 到达时，store 用完整 JourneySnapshot 覆盖 snapshot，更新 lastReady，并把 connected 置为 true。连接错误持续 20 秒后启用 fallbackActive，每 15 秒轮询 /state；SSE 恢复时清理轮询。图 3-4 按实际时序表示该回退机制。

图3-4 行程仪表盘加载与实时更新顺序图（当前实现）

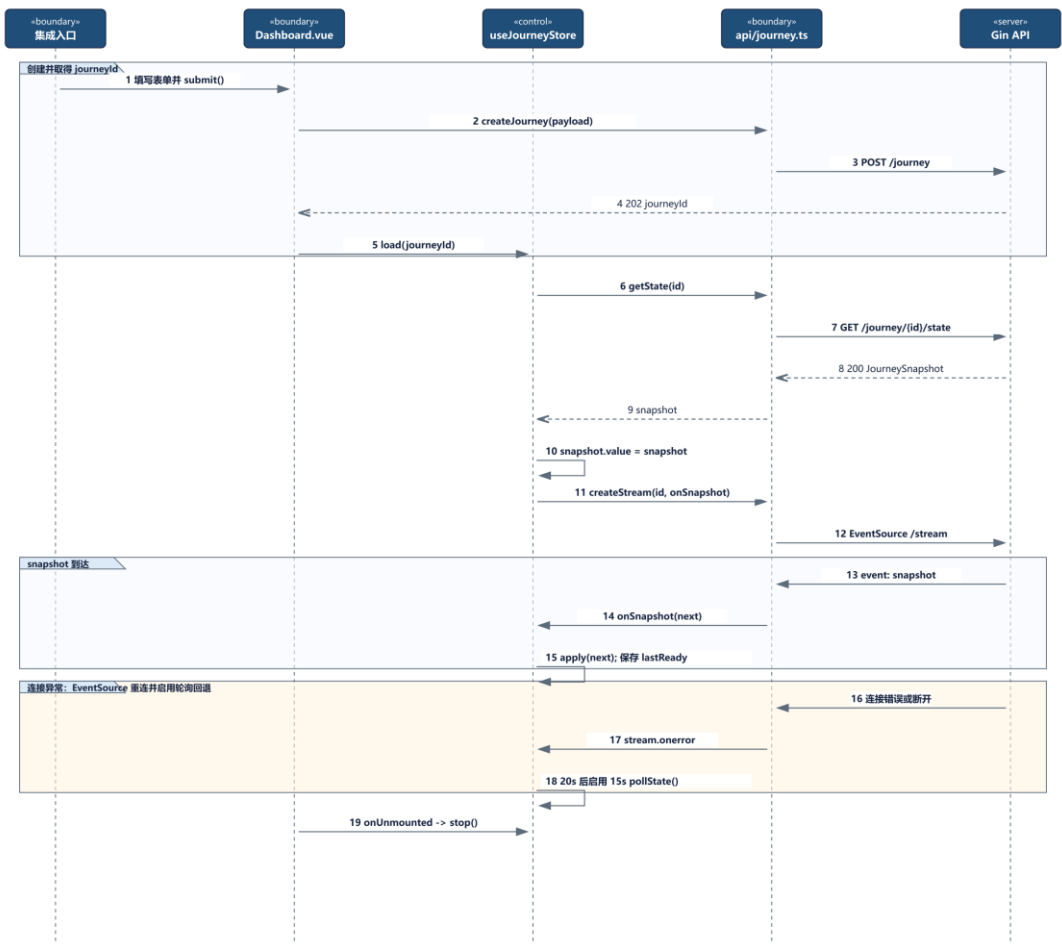


图 3-4 行程仪表盘加载与实时更新顺序图

3.3 界面可用性与异常设计

- 信息层级：风险、下一行动和关键时限优先，来源与解释可在下方查看。
- 响应式：桌面端双栏展示时间轴和行动卡，窄屏改为单栏并扩大按钮触控面积。
- 无障碍：风险不能只靠颜色区分，必须同时显示文字；按钮具有明确标签和键盘焦点。

定位失败：useLocation.error 保存浏览器错误，StageConfirm 仍提供六种手动阶段作为替代入口；系统不会因定位不可用而阻断行程分析。

SSE 失败：EventSource 自动重连，同时 store 在持续断开 20 秒后启动每 15 秒一次的 /state 轮询；连接恢复后关闭回退轮询。

时间显示：FreshnessBar.formatTime 只格式化 state.updatedAt；Timeline.activeIndex 直接使用浏览器 Date.now 与节点时间比较。当前未

实现服务端时钟偏移校正。

第 4 章 详细设计

4.1 用例设计

用例	接口	当前实现对象/函数	结果
创建行程	POST /api/v1/journey	createJourney JourneyHandler.Create Runtime.CreateJourney	202 + processing ; 后台 Recalculate
更新位置/阶段	POST /api/v1/journey/{id}/location	useLocation、updateLocation、 JourneyHandler.UpdateLocation、 Runtime.ApplyLocation	202 + processing; 完成后 SSEHub.Publish
查询当前快照	GET /api/v1/journey/{id}/state	getState JourneyHandler.State Runtime.GetSnapshot MemoryStore.GetSnapshot	200 + 完整 JourneySnapshot
订阅实时状态	GET /api/v1/journey/{id}/stream	createStream JourneyHandler.Stream Runtime.Subscribe、SSEHub	snapshot / heartbeat; 浏览器自动重连

4.1.1 “创建行程”用例实现的设计方案

JourneyHandler.Create 对请求体执行 ShouldBindJSON 和 CreateJourneyRequest.Validate。Validate 只检查 flights 非空以及每个 Flight 的 number、date、from、to 非空；IATA 正则和日期格式由 OpenAPI 契约描述，但当前 Go 校验尚未逐项执行。服务端使用 newJourneyID 生成不透明 ID，调用 Runtime.CreateJourney 后返回 202。

Runtime.CreateJourney 保存 Journey 与零值 JourneyProgress，通过 publishProcessing 生成并广播 processing 快照，然后由 analyzeAsync 调用 Recalculate。Recalculate 用 busy.LoadOrStore 去重，依次调用 data.Agent.BuildState 和 advice.Agent.Evaluate，最后由 finish 原子保存并通过 SSEHub.Publish 广播 ready 或 failed 快照。

图4-1 创建行程与异步分析顺序图（当前实现）

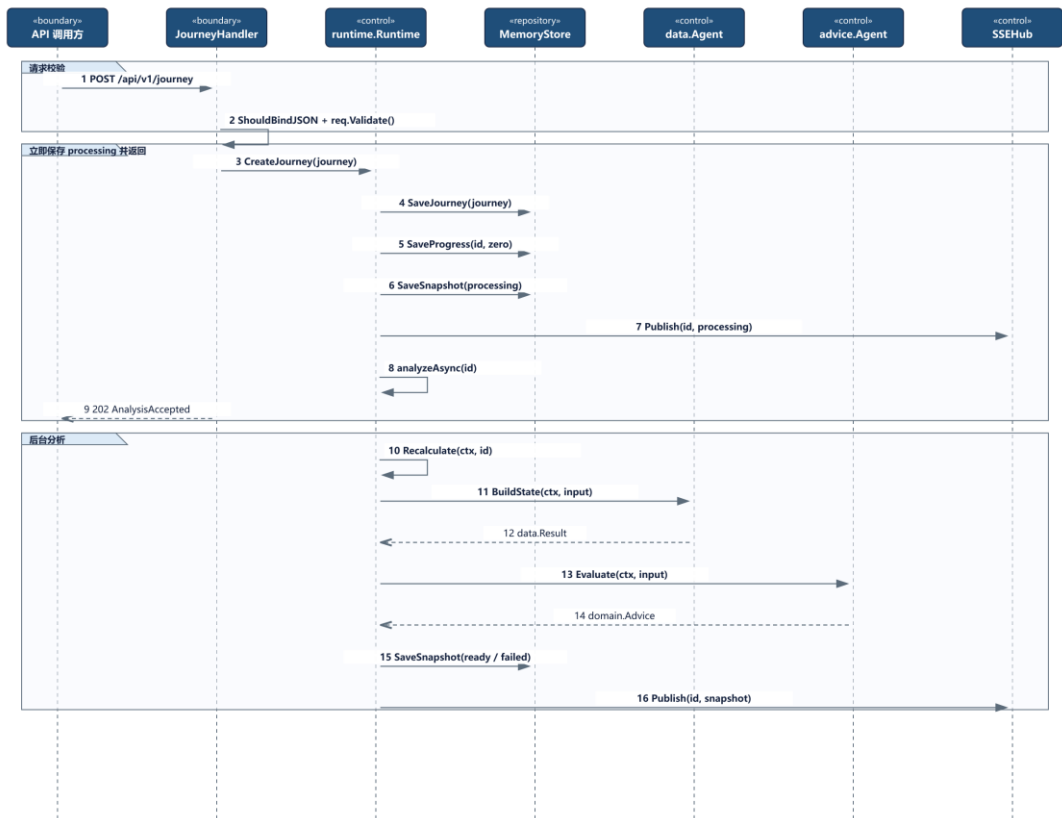


图 4-1 创建行程与异步分析顺序图

4.1.2 “更新位置或手动阶段”用例实现的设计方案

UpdateLocationRequest.Validate 要求 lat 与 lng 成对出现，并要求坐标或 stage 至少存在一种；stage 必须属于 ManualStages。JourneyHandler.UpdateLocation 调用 Runtime.ApplyLocation，行程不存在时返回 404。

ApplyLocation 读取现有 JourneyProgress: 坐标存在时写入 Coordinate 并清除 ManualStage, stage 存在时再写入手动阶段。保存进度后立即发布 processing 并启动异步重算。RouteETASkill 按机场坐标计算 Haversine 直线距离，乘 1.3 路网系数并按 30 km/h 估算 ETA，结果标记为低置信度，不等同于实时地图路线。

图4-2 更新位置或手动阶段顺序图（当前实现）

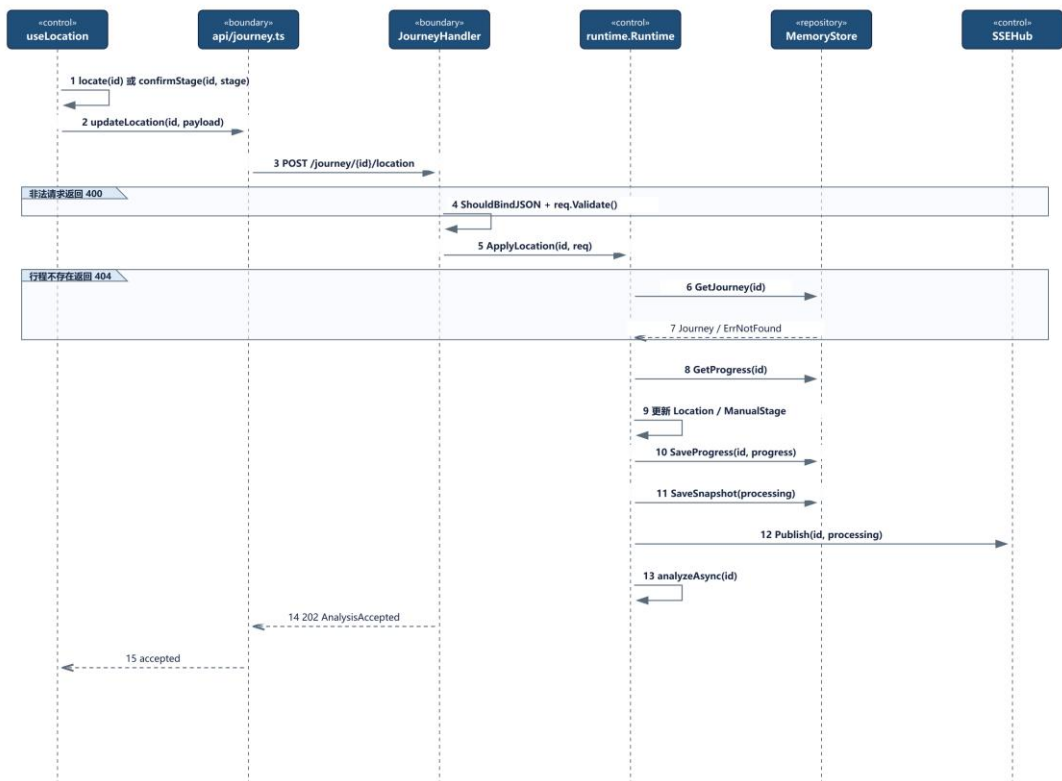


图 4-2 更新位置或手动阶段顺序图

4.1.3 “查询状态与实时订阅”用例实现的设计方案

JourneyHandler.State 调用 Runtime.GetSnapshot 并返回 MemoryStore 中的完整快照，不在 GET 请求中触发分析。JourneyHandler.Stream 先 Subscribe、后 GetSnapshot，避免订阅与读取之间漏掉更新；随后写入 text/event-stream、no-cache、keep-alive、X-Accel-Buffering=no 和 retry: 5000。

Stream 建立后立即发送 snapshot，SSEHub 的后续更新继续发送 snapshot，15 秒无业务变化时发送 heartbeat。前端 createStream 监听 snapshot；浏览器自动重连，应用层在持续断开 20 秒后启动 15 秒轮询回退。

图4-3 查询快照与 SSE 订阅顺序图（当前实现）

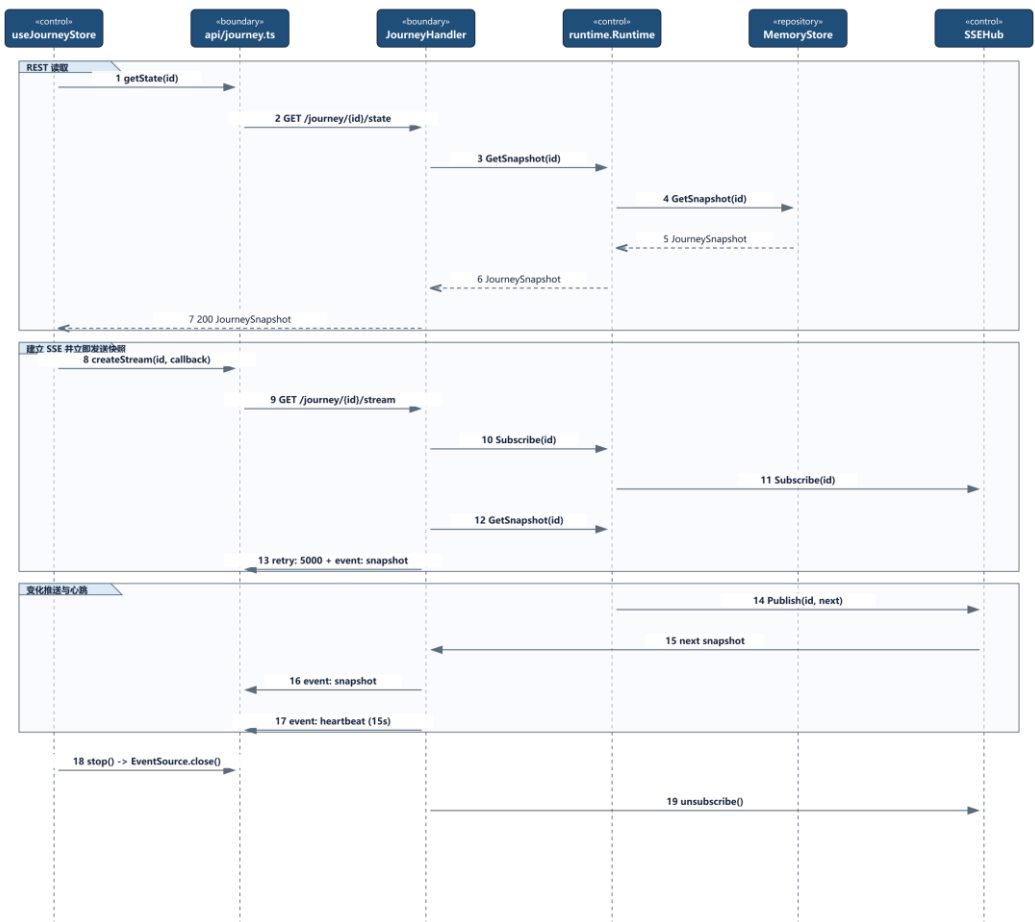


图 4-3 查询快照与 SSE 订阅顺序图

4.2 类设计

图4-4 (a) 接口 编排 存储与推送类图

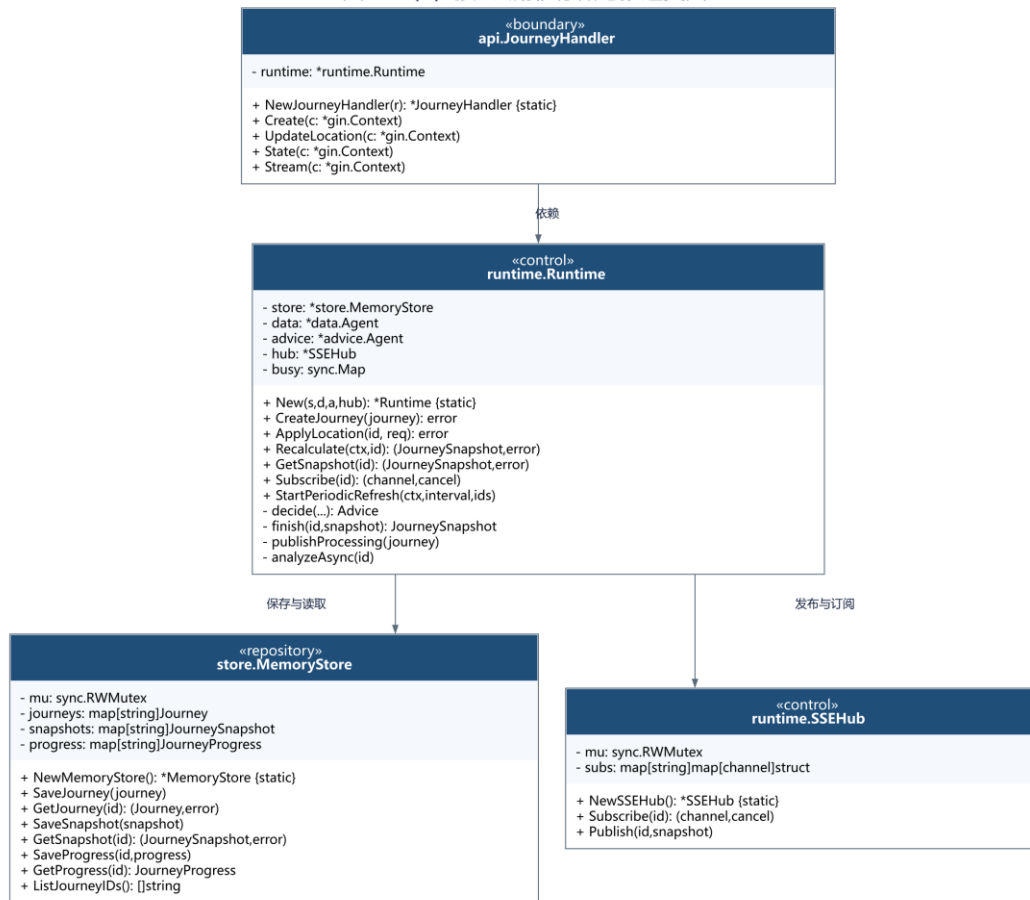


图4-4 (b) Agent 技能与模型接口类图

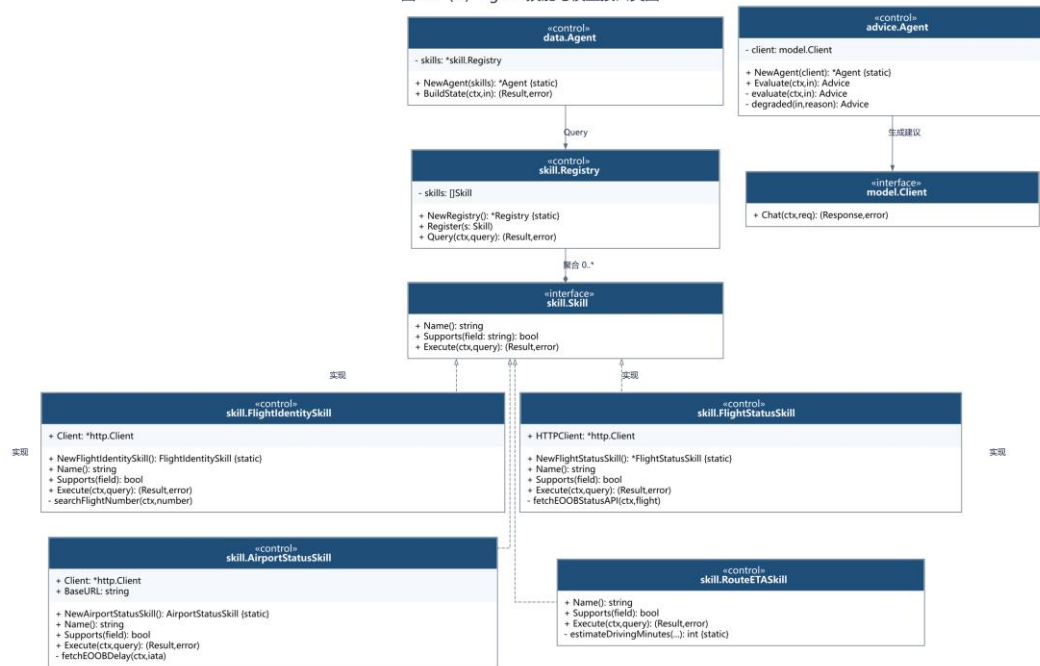


图 4-4 后端实现设计类图

设计类/模块	UML 类别	核心属性	实际公开操作 (+) 与关键私有操作 (-)
api.JourneyHandler	边界类	- runtime	+ Create / UpdateLocation / State / Stream
runtime.Runtime	控制类	- store / data / advice / hub / busy	+ CreateJourney / ApplyLocation / Recalculate / GetSnapshot / Subscribe / StartPeriodicRefresh; - decide / finish / publishProcessing / analyzeAsync
data.Agent	控制类	- skills	+ BuildState; - applyObservations / assessQuality
advice.Agent	控制类	- client	+ Evaluate; - evaluate / degraded
skill.Registry	控制类	- skills[]	+ Register / Query
skill.Skill	接口类	—	+ Name / Supports / Execute
store.MemoryStore	仓储类	- mu / journeys / snapshots / progress	+ SaveJourney / GetJourney / SaveSnapshot / GetSnapshot / SaveProgress / GetProgress / ListJourneyIDs
runtime.SSEHub	控制类	- mu / subs	+ Subscribe / Publish
useJourneyStore / useLocation	前端控制模块	snapshot / lastReady / connected / fallbackActive /	+ load / stop; - apply / pollState / clearFallback; + locate / confirmStage

设计类/模块	UML 类别	核心属性	实际公开操作 (+) 与关键私有操作 (-)
		timers: granted / error	

图 4-4 分为两个子图，以保证方法签名在 Word 页面中仍可读。Go 类型的导出字段和导出方法以“+”表示，未导出字段和私有辅助方法以“-”表示；New、NewAgent、NewRegistry 等包级构造函数以“{static}”标识。data.Agent 与 advice.Agent 在不同包中同名，因此使用包限定名消除歧义。

4.2.1 DataAgent 详细设计

data.Agent 的输入 data.Input 包含 Journey 和 JourneyProgress。BuildState 先在航班身份不完整时调用 FlightIdentitySkill 解析航段，再由 data.MissingFields、skill.Registry.Query、applyObservations 和 assessQuality 构造结果，输出 data.Result{State, Journey, Issues}，使补全后的 Journey 同步进入快照。

skill.Observation 包含 Field、Value、Source、ObservedAt 和 Confidence。FlightIdentitySkill 调用 EOOB 航班搜索 JSON，FlightStatusSkill 调用 EOOB 状态 JSON，AirportStatusSkill 解析 EOOB 机场 HTML 及其中的客流/延误数据，RouteETASkill 执行本地距离估算；前三者使用 5 分钟缓存或受控 HTTP Client，单项失败转化为 issue 而不伪造观测值。

4.2.2 AdviceAgent 详细设计

advice.Agent.Evaluate 先调用私有 evaluate。模型未配置、输入构造失败、模型调用失败或枚举校验失败时，degraded 返回 risk=unknown 的空建议；手动阶段存在时覆盖模型返回的 Stage；最后由 enforceRiskFloor 执行代码级风险下限。

课程 PPT 提出的 $bufferMin = latestDeparture - serverNow - etaMin$ 是目标规则，但当前代码尚未实现独立的确定性缓冲计算。当前 enforceRiskFloor 按 Stage 检查航班状态、时间轴和 ETA，关键数据缺失时把 Risk 设为 unknown，并过滤带 Nav 的行动，防止数据不足时给出高强度导航建议。

模型输出先进入 modelAdvice，再由 valid、toDomain 和 cleanStringPtr 做结构清洗。Action 的导航意图只允许模型写符号值 airport，实际 Nav 由 BuildAirportNav

根据机场 IATA 生成；无法识别机场时返回 nil。

4.2.3 Runtime 与并发控制

Runtime 不维护每个行程的互斥锁队列，而用 `sync.Map busy` 标记正在分析的 `journeyId`。并发调用 `Recalculate` 时，首个调用执行分析，后续调用直接返回当前快照。`finish` 先 `SaveSnapshot` 再 `Publish`，使客户端只观察到完整快照。

`StartPeriodicRefresh` 每隔配置周期遍历 `MemoryStore.ListJourneyIDs` 并调用 `Recalculate`；服务关闭时 `context` 取消 `ticker`。`SSEHub` 只发布结果，不触发分析。位置或阶段更新由 `ApplyLocation` 单独触发，不等待周期刷新。

4.3 数据模型设计

4.3.1 航旅智行系统数据设计类图

对外数据模型严格采用 OpenAPI 契约。`JourneySnapshot` 聚合 `Journey`、`State` 和 `Advice`；`JourneyProgress` 与 `Coordinate` 是仅后端内部使用的进度对象。课程版本没有数据库，因此图 4-5 使用领域类和 DTO，而不再使用 `T_Journey`、`T_WorldState` 等数据库表名。

图4-5 (a) 快照 行程与状态数据类图

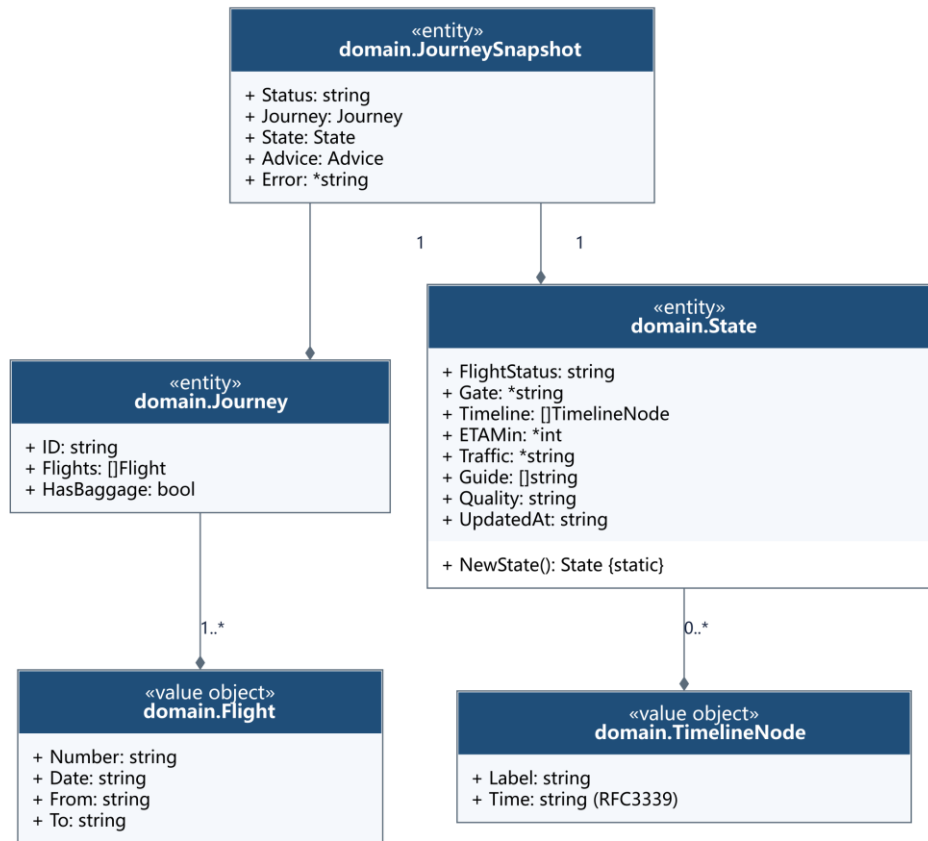


图4-5 (b) 建议数据类图

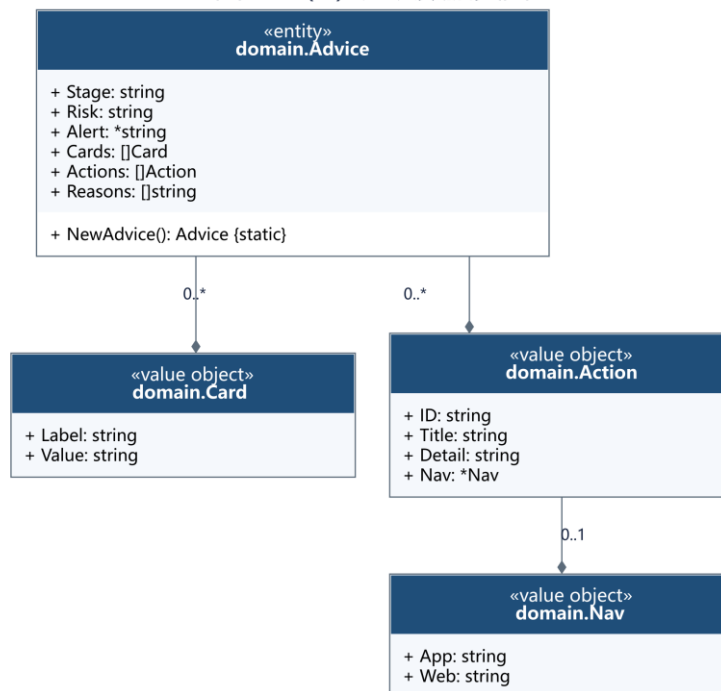


图4-5 (c) 请求对象与内部进度类图

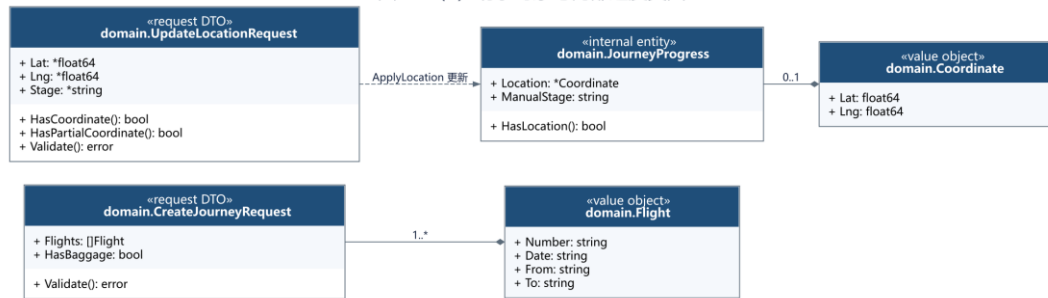


图 4-5 领域与契约数据类图

对象	含义	关键约束
JourneySnapshot	REST 与 SSE 共用的聚合根	Status、Journey、State、Advice 始终输出；Error 仅失败时出现
Journey / Flight	行程与航段	ID 不透明；Flights 至少一个；当前取数只分析首段
State / TimelineNode	DataAgent 输出及时间节点	未知值为 nil/null；Timeline 按 RFC3339 时间升序
Advice / Card / Action / Nav	可直接展示的建议聚合	数组非 null；Nav 可空；链接由 BuildAirportNav 生成
JourneyProgress / Coordinate	后端内部定位与手动阶段	不进入接口响应；坐标上报清除旧手动阶段
CreateJourneyRequest / UpdateLocationRequest	HTTP 请求 DTO	分别提供 Validate；格式级契约校验仍需补齐

图 4-5 分为三个子图：子图（a）描述快照、行程与状态聚合；子图（b）描述 Advice、Card、Action 和 Nav；子图（c）描述请求 DTO、JourneyProgress 与 Coordinate。组合关系标明 1、0..1、0..* 和 1..* 基数。

4.3.2 系统数据的操作设计

MemoryStore 使用一个 sync.RWMutex 保护三个映射：journeys、snapshots 和 progress。SaveSnapshot 以 snapshot.Journey.ID 为键原子替换完整快照；GetSnapshot 不返回内部可变指针。课程版本不持久化，进程重启后数据丢失。

所属类型	操作	输入/输出	设计说明
MemoryStore	SaveJourney / GetJourney	Journey；id -> Journey/error	RWMutex 保护 journeys 映射
MemoryStore	SaveSnapshot / GetSnapshot	JourneySnapshot；id ->	以 snapshot.Journey.ID 为键原子替换

所属类型	操作	输入/输出	设计说明
		JourneySnapshot/error	
MemoryStore	SaveProgress / GetProgress	id + JourneyProgress; id -> JourneyProgress	保存位置和 ManualStage, 零值不报错
MemoryStore	ListJourneyIDs	无 -> []string	返回新切片, 供周期刷新遍历
SSEHub	Subscribe	id -> channel + cancel	缓冲深度 4; cancel 使用 sync.Once 清理订阅
SSEHub	Publish	id + JourneySnapshot	非阻塞发送; 慢订阅者缓冲 满时丢弃该帧

当前代码尚未定义 JourneyRepository 或 SnapshotRepository 接口。若后续需要数据库, 应先抽象仓储接口并让 Runtime 依赖接口, 同时保持 Agent 不直接访问持久化层。

4.3.3 枚举、时间与空值设计

类型	取值或格式
AnalysisStatus	processing、ready、failed
Stage	unknown、en_route、at_airport、check_in、security、waiting、boarding、departed、disrupted
ManualStage	en_route、at_airport、check_in、security、waiting、boarding
Risk	unknown、green、yellow、orange、red
Quality	complete、degraded、unknown
Traffic	unknown、light、moderate、heavy、severe
FlightStatus	unknown、scheduled、on_time、delayed、boarding、departed、cancelled、diverted
时间点	RFC3339 且必须带时区
时长	整数分钟, 字段名以 Min 结尾

4.4 部署设计

当前可运行形态为前后端分离: Vite 开发服务器运行前端, Gin 后端默认运行在 :8080, mockserver 可运行在 :8081。backend/cmd/server 尚未使用 go:embed

提供 frontend/dist, 因此“单二进制同时托管静态页面”仍是 PPT 和项目计划中的部署目标。测试期间因本机 :8080 被其他服务占用, 将真实后端临时运行于 :8081, 并通过既有 Vite 代理完成联调。



图 4-6 系统部署图 (当前可运行链路与外部边界)

系统组成部分	运行环境/外部环境	通信方式	当前实现与配置
Vue 3 SPA	支持 ES2020 、 Geolocation、EventSource 的现代浏览器	HTTP/JSON、 SSE、高德 URI	Vite 独立运行; 已接入创建、定位/阶段、重试和断线轮询
Go 应用服务器	Windows 或 Linux 的 Go 进程	Gin HTTP :8080	Runtime、两类 Agent、MemoryStore、SSEHub 同进程
EOOB 聚合站点	外部航班搜索、航班状态和机场页面	HTTPS / JSON / HTML	标准 HTTP Client; 5 分钟缓存; 失败记录 issue 并安全降级
模型服务与本地 ETA	OpenAI 兼容/百炼原生端点; RouteETASkill	HTTPS / 进程内调用	模型只生成建议; 未配置时 risk=unknown; ETA 不含实时路况
高德导航	旅客设备上的高德 App 或网页	amapuri / HTTPS	仅 NavigateButton.open 在用户点击后打开, 不回传轨迹

4.4.1 部署安全与运维

- 生产环境必须通过 HTTPS 传输定位和航班信息, 外部服务密钥只保存在服务端环境变量或安全配置中。

- 日志记录请求标识、journeyId、处理阶段和错误类型，但不得记录完整精确定位或任何密钥。

- 为外部数据源和模型调用配置独立超时；错误进入 quality/reasons，不能无限重试阻塞旅程。

部署到反向代理后应关闭 SSE 缓冲并设置足够长的读取超时；当前服务端已发送 X-Accel-Buffering=no，并每 15 秒输出 heartbeat。

MemoryStore 不支持多实例共享。当前课程版本保持单个后端实例；若扩容，需要先引入共享持久化与跨实例发布机制。

4.5 接口与契约实现要求

方法	路径	成功码	请求/响应
POST	/api/v1/journey	202	CreateJourneyRequest → AnalysisAccepted
POST	/api/v1/journey/{id}/location	202	UpdateLocationRequest → AnalysisAccepted
GET	/api/v1/journey/{id}/state	200	JourneySnapshot
GET	/api/v1/journey/{id}/stream	200	text/event-stream: snapshot / heartbeat

后端响应必须能够通过 OpenAPI 校验。Go 中可空数值、字符串和时间使用指针并去掉 omitempty；数组初始化为空切片。前端只能从 src/types/index.ts 引入由 OpenAPI 生成的类型，不得使用 as any、@ts-ignore 或手写平行接口绕过契约。

4.6 测试与验收设计

测试层次	当前或建议场景	通过标准
已实现单元测试	config、runtime、advice、flight_status、evidence、model JSON	go test ./... 通过；关键数据缺失不返回 green
Skill 测试	EOOB JSON/HTML 解析、身份补全、缓存与异常降级	无外部依据的字段不得进入 Observation；失败必须记录 issue
Runtime 测试	busy 去重、processing/ready/failed、位置更新	同一行程不积压重复分析；快照顶层完整

测试层次	当前或建议场景	通过标准
API 契约测试	4 个接口、状态码、null/[]、枚举、RFC3339	响应通过 docs/api/openapi.yaml 校验
SSE 测试	首次 snapshot、变化推送、heartbeat、取消订阅	事件名和数据结构符合契约
前端构建测试	Vue/TypeScript 生产构建	npm run build 通过，102 个模块完成转换
端到端集成测试	无头 Edge：创建、processing、EOOB、SSE、阶段确认	9 项断言全部通过；关键接口依次返回 202/200/200/202

提交前执行后端 `go build ./...`、`go vet ./...`、`go test ./...`，前端执行 `npm run build`。修改 OpenAPI 后必须执行 `npm run gen:api`，并确认生成类型已同步提交。

4.7 设计决策与后续扩展

本版已经实现单个 Go 后端二进制、内存存储、双 Agent 编排、EOOB 数据技能、SSE、断线轮询回退和可交互 Vue GUI；前端仍通过独立 Vite 工程运行。设计选择优先保证可运行、可解释和安全降级的 Agent 闭环，不把尚未落地的分布式部署、嵌入式地图或实时道路 ETA 写成已实现。

- 多航段：扩展 State 为当前航段视图并增加衔接评估，但保持 flights[] 和快照整体推送原则。
- 持久化：以 Repository 接口替换 Memory Store，实现进程重启恢复和多实例共享。
- 通知：在明确授权、频率限制和审计后增加独立通知通道，不能复用 advice.alert 充当事件历史。
- 更细粒度溯源：如产品确需逐字段来源和置信度，应先走契约变更流程，避免在现有 reasons 中继续膨胀。
- 真实航司操作：必须新增身份、权限、用户二次确认、费用展示、幂等和审计层，不能直接由当前 Action 执行。

5 集成测试与验证

本章记录 2026 年 9 月 15 日对当前代码进行的构建、前后端集成、EOOB 数据抓取和 UI 验证。UI 使用用户授权的无头 Microsoft Edge（1440×1000 视口）执行；因本机 8080 端口被其他服务占用，测试后端临时运行于 8081，Vite 前端运行于 5173 并通过既有代理访问该真实后端。该端口调整只影响测试环境，生产默认配置仍为 8080。

5.1 构建与自动化测试

测试项	执行内容	结果
Go 后端	go test ./...	通过；全部包编译并完成现有单元测试
Vue 前端	npm ci; npm run build	通过；108 个依赖包、0 个漏洞，102 个模块完成转换
UI 集成	无头 Edge 执行创建、等待就绪、SSE 与阶段确认	通过；9 项断言全部成功
接口链路	POST journey; GET state; GET stream; POST location	依次返回 202、200、200、202

集成脚本直接操作页面表单而非绕过 UI 调用接口，并同时记录浏览器网络响应和控制台错误。测试中仅忽略浏览器请求缺失 favicon 产生的无害 404；业务请求没有失败。

5.2 EOOB 航班抓取验证

选择约 5 个国内航班验证 EOOB 搜索与状态链路。以下状态和计划/实际时间均由 EOOB 结构化接口返回，置信度为 high；登机和登机口关闭时间是系统依据起飞时间分别提前 40 分钟和 15 分钟推导的规则值，不冒充 EOOB 官方字段。

航班/航线/日期	EOOB 状态与时间	耗时	问题与降级
CZ3101 CAN→PKX 2026-09-15	on_time; 08:00–11:05 (+08:00)	2 s	仅登机/关闭时间为推导值
MU5103 SHA→PEK 2026-09-15	departed; 01:00Z–11:15 (+08:00)	2 s	出发机场时区请求 403；保留可验证时间

航班/航线/日期	EOOB 状态与时间	耗时	问题与降级
CA1501 PEK→SHA 2026-09-15	departed; 08:30–10:55 (+08:00)	3 s	仅登机/关闭时间为推导值
HU7603 PEK→SHA 2026-09-15	departed; 08:45–10:55 (+08:00)	12 s	仅登机/关闭时间为推导值
3U8881 CTU→PEK 2026-09-18	scheduled; 13:30Z–16:05Z	2 s	机场时区请求 403; 保留 UTC

五个样本均成功解析航班身份、航线、状态与时间。机场时区端点出现 403 时，系统没有猜测本地时区，而是保留 EOOB 可验证的 UTC 时间并在 reasons/issues 中显式说明，符合“证据不足时安全降级”的设计原则。

5.3 UI 联调过程与截图

代表性用例 CZ3101（CAN→PKX，2026-09-15）依次覆盖创建表单、异步处理中、EOOB 数据展示、SSE 连接及手动阶段确认。截图均来自同一次自动化测试会话。

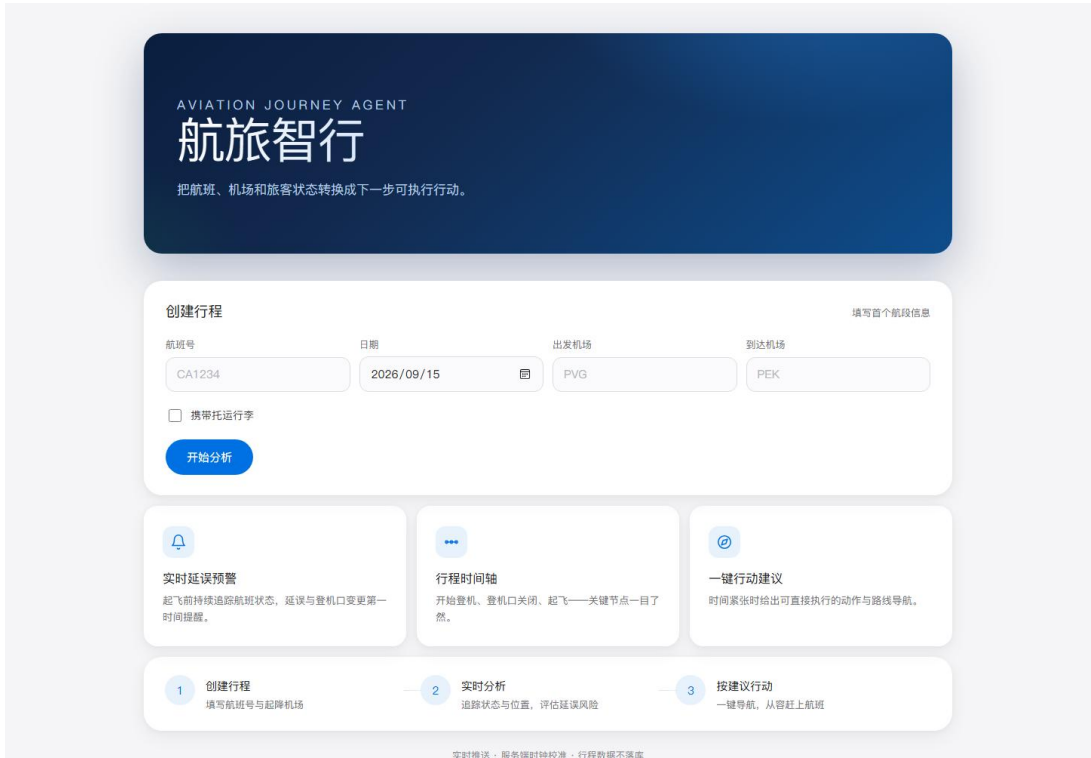


图 5-1 创建行程界面



图 5-2 行程分析处理中界面

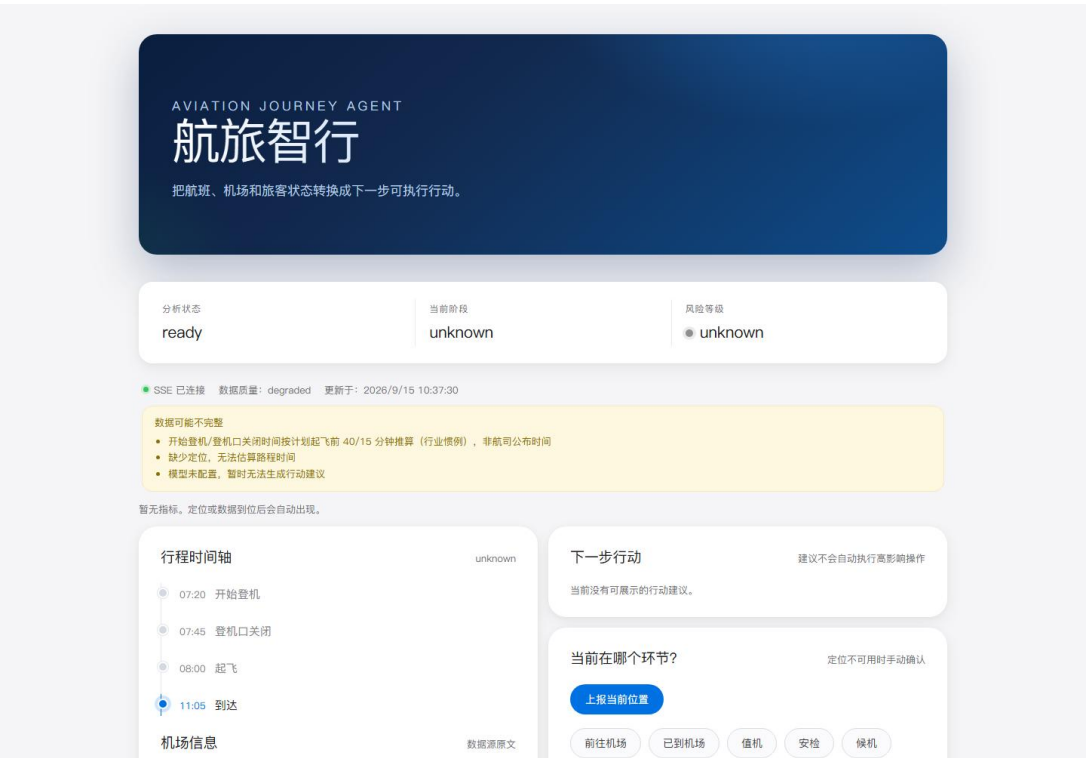


图 5-3 EOOB 数据与 SSE 就绪界面



图 5-4 手动阶段确认后的界面

5.4 测试结论与已知问题

测试确认“页面创建行程→后端异步取数→EOOB 状态返回→完整快照渲染→SSE 更新→手动阶段重算”的前后端闭环可复现。当前测试环境未配置模型 API Key，因此 AdviceAgent 按设计返回 risk=unknown 与空行动列表，并显示“模型未配置”等降级原因；这不是接口或 UI 故障。

仍需改进的界面问题包括：ready、unknown、at_airport 等后端枚举当前直接显示英文值，应在正式展示前增加中文映射；项目未提供 favicon，浏览器会产生一个不影响业务的 404；嵌入式地图和基于实时路况的 ETA 尚未实现。上述事项均已在设计边界中保留，不计入本次已通过能力。